

Introduction to Loops

Without Loops (Redundant Code)

```
// To print "hello" 5 times:
```

```
console.log("hello")
```

```
console.log("hello")
```

```
console.log("hello")
```

```
console.log("hello")
```

```
console.log("hello")
```

Manually repeating code is inefficient and prone to errors.

With Loops (Efficient Solution)

Loops automate repetitive tasks using three key components:

- **Initialization:** Set start value (e.g., let $i = 0$)
- **Condition:** Stopping point (e.g., $i < 10$)
- **Update:** Increment/Decrement (e.g., $i++$)

```
for(let i = 0; i < 10; i++) {  
  
  console.log("hello")  
  
}
```

The while Loop

How it Works

The **while** loop executes a block of code as long as the specified condition evaluates to true.

Key Components:

- **Initialization:** `let j = 1`
- **Condition:** `j < 4`
- **Update:** `j++` (Increments `j` by 1 each time)

// Code Implementation

```
let j = 1;

while(j < 4) {

    console.log(j);

    j++; // post increment operator

}
```

```
console.log("value of j is ", j);
```

// Expected Output:

1, 2, 3

value of j is 4

The do-while Loop

How it Works

The **do-while** loop is similar to the while loop, but it guarantees the code block will execute **at least once** before the condition is tested.

Key Components:

- **Initialization:** `let m = 1`
- **Execution:** Runs the code block first.
- **Condition:** `m < 5` (checked after)
- **Update:** `m++`

// Code Implementation

```
let m = 1;
do {
  console.log("Hey there");
  m++;
} while(m < 5);
```

// Output: Prints "Hey there" 4 times

The for-of Loop

How it Works

The **for-of** loop is used to iterate over iterable objects like strings and arrays.

Key Benefits:

- Simple syntax for iterating values.
- Works directly on characters in a string.
- No need for manual index management.

```
// Iterating over a string
```

```
let str = "Vande Mataram";  
for(let i of str) {  
    console.log(i);  
}
```

```
// Output: Prints each character of the  
string on a new line.
```

The for-in Loop

How it Works

The **for-in** loop is used to iterate over the properties (keys) of an object.

Key Characteristics:

- Retrieves the **keys** of the object one by one.
- Commonly used for objects, not recommended for arrays.
- Access values using `object[key]` syntax.

```
// Iterating over an object
```

```
let student = {  
  name : "Jay Shah",  
  age : 22,  
  cgpa : 9.2,  
  isPass : true  
}
```

```
for(let i in student) {  
  console.log("key is :", i,  
    "& value is :", student[i]);  
}
```

Strings in JavaScript

Concept

A string is a sequence of characters used to represent text content.

Key Properties:

- **Length:** Use `.length` to get the total characters.
- **Indices:** Access characters using `str[0]` (Zero-based).

```
// Variable Declaration
let str1 = "Sarvarth";

// Check length
console.log(str1.length); // Output: 8

// Access indices
console.log(str1[0], str1[1], str1[2]); //
Output: S a r
```

Template Literals

Key Concepts

Template literals provide an easy way to interpolate variables and expressions into strings.

Features:

- Backticks: Use ``` instead of quotes.
- Interpolation: Embed variables using `${}`.
- Readability: Makes complex string building much cleaner.

```
// Template Literals Example  
  
let sentence = `student name is  
${student.name},  
  
He is a good student and he is just  
  
${student.age}. He has CGPA of  
${student.cgpa}.  
  
Is he pass ? ${student.isPass}`  
  
// String interpolation & placeholders
```

Escape Characters

Key Concepts

Escape characters allow you to include special characters in strings that would otherwise be difficult to type.

- **\n**: New Line character. Moves following text to the next line.
- **\t**: Tab character. Adds a horizontal tab space.
- **Length**: Escape characters count as 1 character in string length.

```
// New Line Example
let sentence2 = "hello \nworld";
console.log(sentence2); // Output: hello (new
line) world

// String Length with Escape Chars
console.log(sentence2.length); // Counts \n as
one char

// Tab Space Example
console.log("hello \tworld");
// Output: hello world
```


String Methods in JS

Core Methods

These are built-in functions in JavaScript used to manipulate strings without changing the original string.

- **toUpperCase():** Converts all characters to uppercase.
- **toLowerCase():** Converts all characters to lowercase.
- **trim():** Removes whitespace from both ends of a string.

```
// Example Usage  
let msg = " Hello World ";  
  
console.log(msg.toUpperCase()); // " HELLO  
WORLD "  
  
console.log(msg.toLowerCase()); // " hello  
world "  
  
console.log(msg.trim()); // "Hello World"  
  
// Note: Original string remains unchanged
```

String Methods in JS (Part 2)

Advanced Methods

These methods allow for more specific string manipulation like extracting, joining, and replacing content.

- **slice(start, end):** Returns a part of the string.
- **concat(str2):** Joins two or more strings.
- **replace(old, new):** Searches and replaces a value.
- **charAt(index):** Returns character at a specific index.

```
// 4. slice()
let str = "0123456";
console.log(str.slice(1, 4)); // "123"

// 5. concat()
console.log(str1.concat(str2));

// 6. replace()
console.log(str.replace("h", "m"));

// 7. charAt()
console.log(str.charAt(3)); // "3"
```

Practice Question

1. Email Generator

Your task is to take a user's full name and generate a professional email address following these specific transformation steps:

- Convert all characters to **lowercase**
- Completely **remove all spaces**
- Append **@gmail.com** to the end

Example:

Input: "Omkar Potdar"

Output: omkarpotdar@gmail.com

Practice Question

2. Password Validator

Your task is to take a password as input and validate it based on the following security criteria:

- At least **8 characters** long
- Contains at least one **uppercase letter**
- Contains at least one **lowercase letter**
- Contains at least one **digit**
- Contains at least one **special character**

Example:

Input: "Password123!"

Output: **Valid**